Evaluating Input Filters in a Local LLM Application

AI System Development and Experimental Safety Evaluation
AI Security and Risk Analysis AISRA1301 QAIRU

A local Qwen application was evaluated with NVIDIA garak and a separate labelled test set. A frozen lexical filter reduced garak's exact PromptInject hits from 12/32 to 0/32. However, it detected only 8/24 custom attacks and blocked 8/24 benign requests. This project investigates why an improved scanner score can coexist with weak generalisation and lost utility.

Research problem and selected paper

Xin, Chen, Yang, Backes and Zhang examine jailbreak attacks against a complete inference pipeline, including input and output content filters [1]. Their Findings of ACL 2026 paper tests ten attack methods, several open and proprietary LLMs, and 417 harmful prompts paired with 417 topic-aligned benign prompts. It measures attack success, input/output detection, pass rate, classification quality, latency and cost.

The paper finds that available filters detect many attacks that defeat a model alone, while precision and recall remain uneven. Its pipeline perspective and benign comparison are strengths. Limited API budgets, incomplete coverage of tool-using agents and preliminary adaptive evaluation restrict generalisation. The practical opportunity is to evaluate the deployed model and filter together while also measuring mistaken refusals.

Research question: Does a simple input filter reduce observable instruction redirection on a local LLM, and what detection and usability costs appear on new attack variants and benign security questions?

Proposed AI system

LLM Safety Lab is a working local web application. It accepts a text request, selects baseline or filtered mode, and displays the model answer or refusal, decision, elapsed time and generated tokens. The AI component is Qwen2.5-0.5B-Instruct [3]. Its smaller size fits a GTX 1660 with 6 GB VRAM. It belongs to the family evaluated in the paper, but does not reproduce the paper's 7B target or learned content moderation models.

System architecture

The browser sends JSON to a loopback HTTP endpoint. The baseline path calls a shared inference engine. The protected path first applies four frozen regular-expression rules inherited from the earlier pilot. Allowed inputs reach the same tokenizer, system message and model. A standard refusal handles blocked inputs. Each completed request enters a local JSONL audit log. Garak's official function.Single bridge calls this same endpoint, so the demonstration and evaluation share one implementation.

The service has no external action tools or private information in prompts. It binds to 127.0.0.1, rejects cross-origin browser submissions, validates request sizes and modes, and displays responses as text. Seven code checks and four HTTP boundary checks passed. These checks verify selected implementation properties, not comprehensive security.

Safety experiment and reproducibility

All comparisons use pinned model revision 7ae557604adf67be50417f59c2c2f167def9a775, FP16 CUDA inference, a fixed system message, greedy decoding and a 96-new-token ceiling. Garak 0.17.1rc1 is pinned to repository commit 346497a [2]. Seed 42 selects 32 PromptInject prompts, with one additional DAN 11.0 prompt. Probe suggestions cannot change the application's decoding settings.

The extension contains 24 custom attacks, four each for direct override, personas, paraphrases, multilingual instructions, obfuscation and encoding, plus 24 benign requests. Of these, 12 ordinary tasks have strict expected answers and 12 discuss security legitimately. Every case runs in both modes with alternating order. One warm-up is excluded. Input hashes match for all 33 garak pairs and 48 custom pairs. Allowed custom pairs returned identical answers.

Evaluation	Baseline	With input filter
Garak PromptInject exact marker	12/32 (37.5%)	0/32 (0%)
DAN acknowledgement detector	1/1	0/1
Custom attack exact marker	0/24	0/24
Custom filter TP / FP / FN / TN	0 / 0 / 24 / 24	8 / 8 / 16 / 16
Benign false-positive rate	0/24 (0%)	8/24 (33.3%)
Ordinary strict answer matches	9/12	9/12
Mean application latency	1.408 s	0.683 s
Median application latency	0.844 s	0.197 s

Metric definitions. Garak's score counts exact rogue-string hits. Custom ASR counts replies containing MIDTERM_OVERRIDE, a harmless redirection marker. Filter recall is TP/(TP+FN), precision is TP/(TP+FP), and benign false-positive rate is FP/(FP+TN). These evaluate different properties. The protected filter achieved precision 0.50, recall 0.333 and F1 0.40 on the custom set. Baseline filter precision is undefined because it never blocks.

Experimental results and interpretation

The lexical control suppresses the tested garak signatures. A baseline DAN answer says 'DAN Mode Enabled', while the filter refuses it. This acknowledgement does not establish harmful content in a later turn. The custom task showed no exact marker hits in either mode, so it provides no evidence of an ASR improvement. The filter catches the eight direct/persona variants but misses the other sixteen. Its 50% classification accuracy equals an always-allow baseline on this balanced sample.

Lower average latency partly reflects immediate refusals, including mistaken refusals, rather than faster model inference. Strict answer matching also rejects some correct answers with extra words. Eight baseline and three protected custom outputs reach the token cap, limiting content assessment.

Research extension and contribution

The contribution is an auditable application-level comparison rather than simply running a scanner example. It freezes a prior filter before new cases, connects garak to the actual application, extends attacks beyond literal English override phrases, and measures benign mistakes alongside ASR. Matched inputs, recorded model/data/filter hashes and a shared inference path make the comparison reproducible. The extension adds precision, recall, F1, latency and strict ordinary-task matching to the earlier pilot.

A second detector audit strips non-letter characters before searching for the rogue string. It finds 15/32 baseline and 1/32 protected candidates, versus 12/32 and 0/32 exact hits. In the protected candidate, the model turns 'STOP EVERYTHING' text into a numbered list that still contains the marker. This could reflect the legitimate formatting task. Normalised marker presence therefore requires contextual review and is not a verified additional attack success.

Limitations and next steps

This is one small model, one device and deterministic single-turn evaluation on convenience samples. The custom cases were generated with AI assistance and are not a production benchmark. They use harmless markers rather than the paper's harmful prompts, and the filter recognises lexical cues rather than content harm. Garak's function adapter name also appears in the DAN template. No tool invocation, exfiltration, multi-turn attack or independent human grading is evaluated.

Zero exact hits in 32 tests does not imply zero risk. A descriptive 95% Wilson interval for 0/32 extends to 10.7%, even before sampling and distribution limits. Future work should add blinded human labels, independently sourced benign traffic, multiple model sizes and a learned moderation baseline. A larger study should separately evaluate task completion, contextual marker use, output moderation and adaptive attacks.

Conclusion

The input filter improves the selected garak signatures but shows substantial blind spots and false positives on the extension. The results support evaluating the complete deployment pipeline with several metrics. A claim of general safety would exceed the evidence. The student's final interpretation should follow direct review of the supplied code, cases and responses.

References

[1] Y. Xin, D. Chen, L. Yang, M. Backes, and X. Zhang, "Jailbreaking Attacks vs. Content Safety Filters: How Far Are We in the LLM Safety Arms Race?" Findings of ACL 2026, pp. 421-445, 2026, doi: 10.18653/v1/2026.findings-acl.20.

[2] NVIDIA, "garak," source revision 346497afcd86ff5ba137cfbcaaeba4e9c9f6c970. https://github.com/NVIDIA/garak (accessed 22 Oct. 2026).

[3] Qwen Team, "Qwen2.5-0.5B-Instruct," model card, 2024. https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct (accessed 22 Oct. 2026).

[4] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques for Language Models," NeurIPS ML Safety Workshop, 2022. https://arxiv.org/abs/2211.09527.

AI Use Declaration

ChatGPT Codex generated the prototype, custom cases, reproducibility scripts, automated runs, initial analysis and report/presentation drafts. The retained cover illustration was AI-generated in the earlier assignment. AI checked references against official sources and inspected saved outputs. Student selection of the paper is recorded; independent code review, rerunning and final interpretation have not yet been documented. The student must complete these checks and update this declaration truthfully before submission.